

LegalEase - AI-Powered Legal Document Generator

LegalEase - AI-Powered Legal Document Generator

Government Arts and science college-Manur-MSU142

Team leader:

Nithyakala p NM ID:23FAA3BBED5DBA0932A413567D171184

Team members: Esakki priya p NM ID:77199CD4EDE1276B9514C63C27537B04

Praba Nanthini N NM ID:3CC256046670873B4DD4E3FF32B713BA

*7. PROJECT DOCUMENTATION PHASE

Project Title: LegalEase - AI Powered Legal Document Generator

7.1 Project Synopsis

LegalEase leverages state-of-the-art Generative AI (Gemini 1.5 Pro) to simplify creation of legal documents. Users can generate employment contracts, lease agreements, NDAs tailored to specific inputs like parties involved, effective dates, and key terms. Built with FastAPI and Streamlit, it provides professional-grade documents with custom branding (logo, fonts), editable previews and automatic term tables with export options (.PDF, .DOCX, .TXT). It bridges gap between accessibility and legal professionalism.

7.2 Objective of Documentation Phase

To provide complete written record of project for evaluation, future reference and deployment.

7.3 System Requirements

- *Software:* Python 3.10+, VS Code, Browser, Internet
- *Libraries:* `fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv`
- *Hardware:* i3 Processor, 4GB RAM minimum
- *API Key:* Gemini API Key stored securely in `.env` file

7.4 Technology Stack & Architecture

- *Frontend:* Streamlit - User input, preview, download
- *Backend:* FastAPI (main.py, http://routes.py) - handles API requests
- *AI Core:* `ai\_core/gemini\_generator.py` - Gemini 1.5 Pro integration
- *Architecture Diagram:*

User Input (Streamlit) -> FastAPI POST /generate -> GeminiDocumentGenerator -> Gemini API -> Sanitized Text -> format_docx/format_pdf/format_html_preview -> Download

7.5 Core Implementation Details (with Coding)

A) http://main.py - API Initialization

```
from fastapi import FastAPI
from legalEaseAPI.routes import router
app = FastAPI(title="LegalEase - AI Legal Document Generator")
app.include_router(router)
@app.get("/")
def home():
    return {"message": "Welcome to LegalEase API"}
```

B) http://routes.py - Generation Logic

```
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator
class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

```
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    gen = GeminiDocumentGenerator()
    text = gen.generate_document(request.document_type, request.parties, request.terms,
request.dates)
    return {"document": text}
```

C) gemini_generator.py - Core & Formatting

```
import os, google.generativeai as genai
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
self.model = genai.GenerativeModel("gemini-1.5-pro")
prompt = f"Generate professional {doc_type} between {parties} with terms {terms} effective {dates}"
```

Extra Functions:

```
sanitize_text(text) - removes special quotes
format_docx(text, doc_type) - logo + Times New Roman + terms table
format_pdf(text, doc_type) - FPDF header/footer centered logo
```

D) Deployment Setup

```
venv\scripts\activate
pip install -r requirements.txt
uvicorn legalEaseAPI.main:app --reload # Backend 8000
streamlit run frontend/app.py # Frontend 8501
```

7.6 Testing

Tested 3 scenarios from doc:

1. Startup founder -> Employment Contract (roles, compensation, confidentiality)
2. Freelancer -> NDA (parties, scope)
3. Landlord -> Lease Agreement (property, tenant details)

Result: Avg time 4-6 sec, proper layout, clean formatting, correct term embedding.

7.7 Challenges Faced

1. PDF logo alignment - Fixed using Pillow resize and FPDF header.
2. Gemini token limit for long documents - Solved via structured prompting.
3. DOCX terms table - Semicolon separated input converted to table.

7.8 Security & Privacy

No permanent storage of sensitive data, API key protected in .env, input sanitization to prevent prompt injection.

7.9 Future Enhancement

Multilingual support (Tamil/Hindi), e-Signature integration, Contract Risk Analyzer, Cloud deployment on Render / Streamlit Cloud, integration with legal databases.

7.10 Conclusion

LegalEase represents transformative step in bridging accessibility gap in legal services through AI. It promotes inclusivity, reduces dependency on costly consultations, and its architecture ensures scalability, data privacy and real-time responsiveness. It is a catalyst for legal empowerment.

—